

Session 2.4

PyTorch, Curve Fitting, and Yield Forecasts

The training loop in code, a curve fitted and broken, and an honest one-month-ahead forecast



Zhigang Feng

What PyTorch Actually Is

tensors

arrays of numbers, as in NumPy

autograd

a record of every operation, so exact derivatives come back

nn.Module

a class that holds a network's parameters

optim

gradient-descent variants: SGD, Adam

- ▶ A scientific-computing library that became popular for deep learning, not the reverse.
- ▶ The first two pieces are the substance: write any scalar as a chain of array operations and its gradient is free.
- ▶ The last two are bookkeeping for parameters and update rules.
- ▶ **Everything in Deck 2.3 is four library calls. Today we make them.**

Source: A. Paszke et al., "PyTorch: an imperative style, high-performance deep learning library", *NeurIPS* 32, 2019.

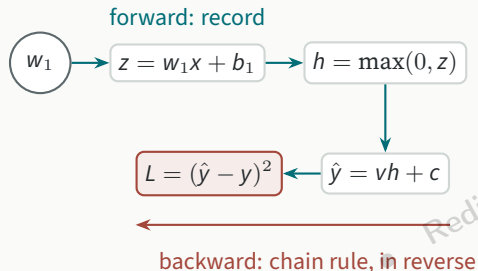
Tensors: Shapes, Types, and the Broadcasting Bug

```
k = torch.linspace(0.5, 2.0, 4)      # shape (4,)
K = k.reshape(-1, 1)                 # shape (4, 1): four observations, one feature
print(k.shape, K.shape, K.dtype)     # torch.Size([4]) torch.Size([4, 1]) torch.float32
print((K - k).shape)                  # torch.Size([4, 4]) <- not what you meant
```

- ▶ A **tensor** is an array that knows its **shape** (here 4 rows, 1 column) and its **dtype**, the number format (32-bit floats by default).
- ▶ **Broadcasting** stretches arrays of different shapes to match. A (4, 1) minus a (4,) silently becomes a 4×4 table, and the loss that follows is wrong without any error message.
- ▶ Convention in this course: a batch of n observations of d variables has shape (n, d) , always.
- ▶ **Most first bugs are shape bugs. Print the shape before you trust the number.**

Source: Session 2 lab notebook, block A1.

Autograd: the Tape



- ▶ Mark a tensor `requires_grad=True` and PyTorch records every operation it enters: the **computation tape**.
- ▶ `backward()` walks the tape in reverse, one chain-rule step per operation: an exact derivative, not a finite difference.
- ▶ This is **autograd**: Deck 2.3's backward pass, done by the library.
- ▶ **You write the forward pass only.**

backward(): Deck 2.3's Numbers, Reproduced

```
x, y = 2.0, 3.0
w1, b1, v, c = (torch.tensor(a, requires_grad=True) for a in (0.5, -0.1, 1.2, 0.3))
z = w1 * x + b1; h = torch.relu(z); yhat = v * h + c; L = (yhat - y) ** 2
L.backward()
# forward : z = 0.9000, h = 0.9000, yhat = 1.3800, L = 2.6244
# backward: dL/dv = -2.9160, dL/dc = -3.2400, dL/dw1 = -7.7760, dL/db1 = -3.8880
# update   : w1 -> 0.5778, v -> 1.2292; loss 2.6244 -> 1.7488      (eta = 0.01)
```

- ▶ The four-parameter ReLU example of Deck 2.3, in four lines. Every number on those slides is asserted against autograd in the lab.
- ▶ A second hidden unit with $z = -0.8 < 0$ receives a gradient of exactly 0.0: no signal, no learning.
- ▶ **If your hand derivative and autograd disagree, the hand derivative is wrong.**

Source: Session 2 lab notebook, block A2; example from Miao & Feng, *AI for Economists*, §9.3.

The Training Loop Is Always the Same Five Lines

```
model = nn.Linear(2, 1)
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)
for step in range(300):
    prediction = model(X)                # 1 forward
    loss = ((prediction - y) ** 2).mean() # 2 loss
    optimizer.zero_grad()                # 3 clear the old gradients
    loss.backward()                       # 4 backward
    optimizer.step()                      # 5 update
```

- ▶ Forward, loss, clear, backward, update. A language model with billions of parameters is trained by these five lines.
- ▶ Only line 1 (the model) and line 2 (the loss) change between applications.
- ▶ Fit a curve: L is the squared prediction error. Solve a model: L is the squared Euler residual, which is Deck 2.5.
- ▶ **Once derivatives are free, one algorithm covers both. Learn the loop once.**

Source: Session 2 lab notebook, block A3.

A Network Is a Class

```
class SimpleNet(nn.Module):  
    def __init__(self, hidden=32, act=nn.ReLU):  
        super().__init__()  
        self.net = nn.Sequential(nn.Linear(1, hidden), act(), nn.Linear(hidden, 1))  
    def forward(self, x):  
        return self.net(x)
```

- ▶ `nn.Module` is PyTorch's base class for anything with parameters. `__init__` declares the layers; `forward` says how an input flows through them.
- ▶ `nn.Linear(1, hidden)` is $WX + b$ with W of shape $\text{hidden} \times 1$; `nn.Sequential` composes layers in order.
- ▶ This is $f_{\theta}(x) = W_2 \sigma(W_1 x + b_1) + b_2$ from Deck 2.2, with 32 hidden units.
- ▶ **The class is the specification. Read it as you would read a regression equation.**

Source: Session 2 lab notebook, block A4.

Optimizers, zero_grad, and Seeds

```
torch.manual_seed(1); net = SimpleNet()           # same seed, same initial draw
optimizer = torch.optim.Adam(net.parameters(), lr=3e-3, weight_decay=weight_decay)
optimizer.zero_grad()                             # 3 clear the old gradients
loss = loss_fn(net(X_tr), y_tr); loss.backward(); optimizer.step()
net.eval()                                         # evaluation mode
with torch.no_grad():                             # no tape: nothing to differentiate
    val = float(loss_fn(net(X_va), y_va))
```

- ▶ `parameters()` hands every W and b to the **optimizer**. `weight_decay` is an L2 penalty on the weights: ridge regression, in one argument.
- ▶ Gradients *accumulate* across calls to `backward()`: without `zero_grad()` you descend on the sum of every gradient so far.
- ▶ Initial weights are a random draw and a **seed** fixes it. `eval()` switches layers such as dropout; `no_grad()` turns the tape off when you only want predictions.
- ▶ **Forgetting `zero_grad()` raises no error. It trains the wrong model.**

Source: Session 2 lab notebook, blocks A4 and C.

Start at Version 0, Where You Know the Answer

```
torch.manual_seed(0)
X = torch.randn(200, 2)
y = 1.0 + X @ torch.tensor([[2.0], [-0.5]]) + 0.1 * torch.randn(200, 1)
# ... the five-line loop on nn.Linear(2, 1), 300 steps ...
# OLS      : [ 1.0073  1.9972 -0.5023]
# learned: [ 1.0073  1.9972 -0.5023]
assert np.allclose(ols, learned, atol=1e-3)
```

- ▶ A linear “network” trained on squared error must land on the OLS coefficients. It does, to four decimals.
- ▶ If it had not, the bug would be in the loop, and you would know before touching a real problem.
- ▶ The same habit in Deck 2.5: solve the growth model first where the policy is known in closed form.
- ▶ **Version 0 of any project is the case with a known answer.**

Source: Session 2 lab notebook, block A3.

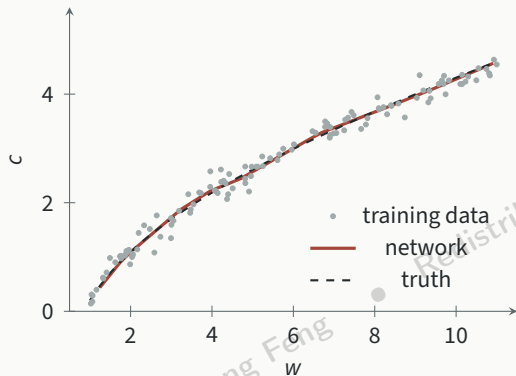
Session 2.4

A Curve Whose Answer Is Known

Fit it, read the residuals, then ask it about a place it has never been



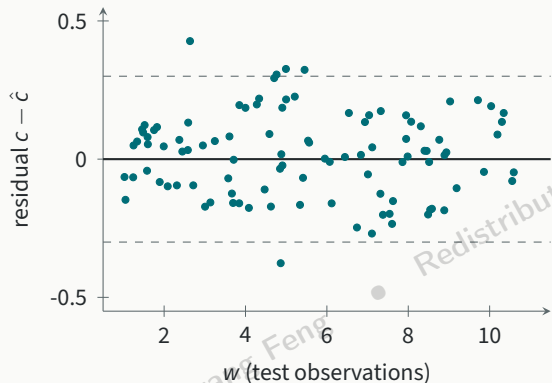
Fit: a Network Finds $c = f(w)$



- ▶ A hidden law, $c = \ln w + 0.2 w + \varepsilon$ on $w \in [1, 11]$, noise variance 0.0225; 300 training and 100 test observations.
- ▶ 32 ReLU units, 1,500 Adam steps. The functional form was never specified.
- ▶ Mean squared error: 0.0231 in training, 0.0229 on the test data.
- ▶ **Both sit at the noise variance: the law was found, the noise was not chased.**

Source: Session 2 lab notebook, block B.

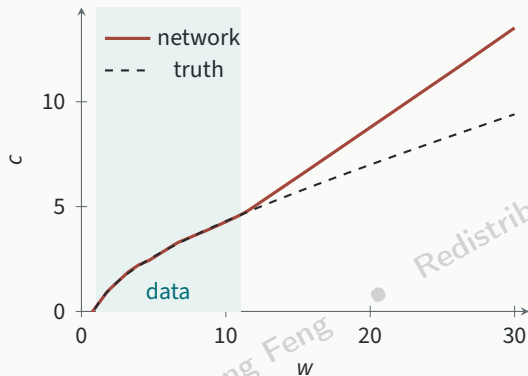
Residuals: What Is Left Should Have No Shape



- ▶ The test residuals scatter evenly around zero across the whole range of w . The dashed lines are two standard deviations of the noise, ± 0.30 .
- ▶ A curve, a fan or a run of one sign in this plot would mean the model missed something systematic.
- ▶ A single number such as the MSE cannot show that. The plot can.
- ▶ **Read residuals exactly as you would after a regression.**

Source: Session 2 lab notebook, block B.

Extrapolation: Outside the Data It Is a Straight Line



- ▶ Ask the fitted network about $w = 30$, far outside the data. Truth: 9.40. Network: 13.52.
- ▶ A ReLU network is piecewise linear. Beyond the last kink it continues its last segment for ever.
- ▶ A tanh network would flatten instead. Neither knows the law; each extends its own shape.
- ▶ **A network interpolates. Outside the training support its answer is an artefact of the architecture.**

Source: Session 2 lab notebook, block B.

Session 2.4

Next Month's 10-Year Yield

One target, one information set, four models, and a result stated in advance



A Frozen Panel: Series, Download Date, Hash

FRED series	What it is	How it enters
DGS10, DGS3MO	10-year and 3-month Treasury yields, daily	last trading day of each month
FEDFUNDS	effective federal funds rate, monthly	value for month $t - 1$
UNRATE	unemployment rate, monthly	value for month $t - 1$
CPIAUCSL	consumer price index, monthly	12-month inflation, month $t - 1$

- ▶ Downloaded once, on 20 September 2026: 439 monthly rows, the last for August 2026.
- ▶ A **hash** (SHA-256) is a file's fingerprint. The notebook stops if the panel's does not match.
- ▶ **Series, date and hash together say exactly which data produced the numbers.**

Source: Federal Reserve Bank of St. Louis, FRED, fred.stlouisfed.org; Session 2 lab notebook, block C.

The Question



- ▶ Standing at the end of month t , forecast the 10-year Treasury yield at the end of month $t + 1$.
- ▶ Train on 1990 to 2014, choose the design on 2015 to 2019, report on January 2020 to August 2026.
- ▶ The test window holds a pandemic and the fastest tightening in forty years.
- ▶ **Can a network beat “no change”?**

Source: FRED series DGS10, month-end, downloaded 20 Sep 2026.

The Target: Month-End, Not the Monthly Average

- ▶ FRED publishes the 10-year yield daily (DGS10) and as a monthly average of those days (GS10). The target here is the **last trading day of the month**.
- ▶ Averaging a persistent daily series over a month builds serial correlation into its monthly *changes*, even when the daily series is a pure random walk (Working, 1960).
- ▶ A model “forecasting” the change in a monthly average is partly predicting that arithmetic, not the market.
- ▶ Month-end values have no such overlap: the change from t to $t + 1$ uses two points in time.
- ▶ The lab asks you to switch to the monthly average and watch the forecast become easier.
- ▶ **Before comparing models, make sure the target does not manufacture predictability.**

Source: H. Working, “Note on the correlation of first differences of averages in a random chain”, *Econometrica* 28(4), 1960.

The Information Set: What Had Been Published by the End of Month t

Predictor	Dated	Why that date
10-year yield	month-end t	a market price, seen the same day
10-year minus 3-month spread	month-end t	market prices
federal funds rate	month $t - 1$	a monthly average
unemployment rate	month $t - 1$	released early in the next month
12-month CPI inflation	month $t - 1$	released in the middle of the next month

- **Information set:** what the forecaster could know. **Publication lag:** the delay between a month and its statistic.
- These are today's revised figures, the current **vintage**, not the real-time ones.
- **Every predictor is dated $\leq t$. A forecast that uses tomorrow's data is not a forecast.**

Splits by Date; Scalers from the Training Window

train 1990–2014 300 months

weights and scaler statistics come from here only

validate

2015–2019
60 months

test

Jan 2020–Aug 2026
79 months

- ▶ Samples are split by the *date of the label*, in order. No shuffle: next month is not a fresh draw.
- ▶ A **scaler** standardises each input to mean 0 and variance 1. Its mean and standard deviation are estimated on 1990–2014 and then applied, unchanged, to the later windows.
- ▶ Validation data choose the width and the stopping epoch. Test data are touched once.
- ▶ **Anything estimated, including the scaler, is estimated on the training window.**

Four Models, One Information Set

Model	Forecast of y_{t+1}	Estimated
random walk	y_t : no change	nothing
AR(1)	$a + \rho y_t$	2 coefficients
OLS	$a + \beta^\top x_t$, all five predictors	6 coefficients
MLP	$y_t + g_\theta(x_t)$, one hidden ReLU layer	width 4: 29 parameters

- ▶ Every model sees the same five predictors, the same splits, the same scoring. Differences in accuracy are differences in the model, nothing else.
- ▶ The network predicts the *change*. A network that outputs zero is the random walk, so it can only lose by learning something false.
- ▶ **A comparison is fair only when the information sets are identical.**

Said in Advance: the Random Walk Is Hard to Beat

- ▶ A long yield is an asset price. If next month's change were easy to predict, traders would already have moved today's price.
- ▶ The forecasting literature agrees: for interest rates and exchange rates at short horizons, “no change” beats most models out of sample.
- ▶ The test window lies partly outside the training support: 2020 brought unemployment never seen in 1990–2014, and 2022–2023 the fastest tightening in forty years.
- ▶ The curve at $w = 30$ showed what a network does outside its data: it extends its last segment.
- ▶ So the expected result is that the network does *not* win. We say so before looking.
- ▶ **If the network loses to “no change”, that is the lesson of the exercise, not its failure.**

Source: R. Meese and K. Rogoff, “Empirical exchange rate models of the seventies”, *JIE* 14, 1983; G. Duffee, “Forecasting interest rates”, *Handbook of Economic Forecasting* 2A, 2013.

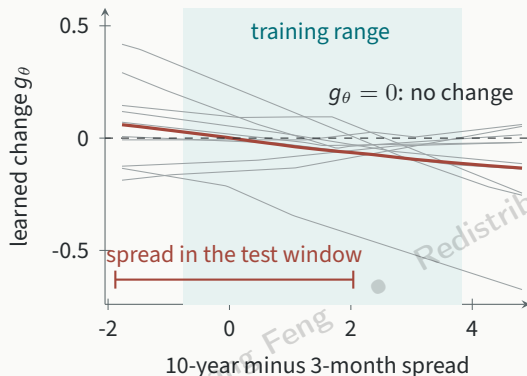
Results, January 2020 to August 2026

Model	RMSE	MAE
random walk	0.2658	0.2097
AR(1)	0.2647	0.2089
OLS, five predictors	0.2855	0.2255
MLP, width 4, mean of 10 seeds	0.3078	0.2406

- ▶ **RMSE**: root mean squared error. **MAE**: mean absolute error. Percentage points, 79 months.
- ▶ The random walk and AR(1) tie. Macro predictors *raise* the error; a network raises it more.
- ▶ Width 4 won on validation: MSE 0.0349, against 0.0469 (width 8) and 0.0403 (16).
- ▶ **More flexibility bought a worse forecast, exactly as announced.**

Source: Session 2 lab notebook, block C; FRED data downloaded 20 Sep 2026.

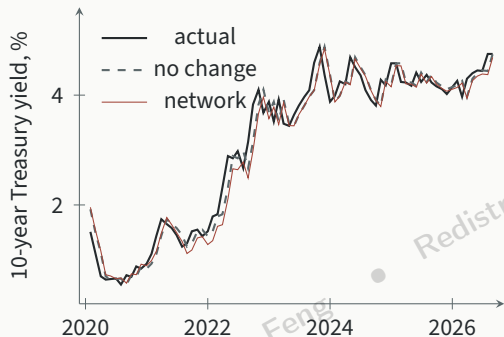
What the Network Actually Learned



- ▶ It predicts the *change*, $\hat{y}_{t+1} = y_t + g_\theta(x_t)$: a network that outputs zero is the random walk.
- ▶ Ten seeds disagree by 0.611 inside the training range and 0.735 at the edges, against a whole error of RMSE 0.2658.
- ▶ Outside that range the ReLU segments simply continue straight, as at $w = 30$.
- ▶ **The seeds disagree by as much as the quantity being forecast.**

Source: Session 2 lab notebook, block C: the width-4 network's learned change, other predictors at their training means.

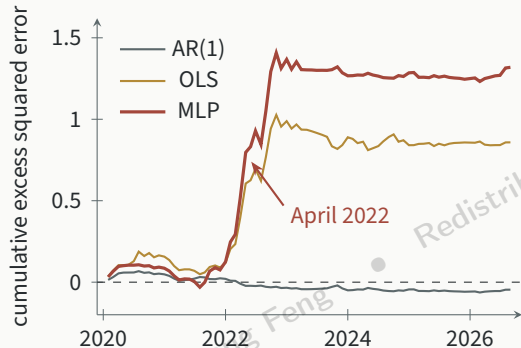
Actual and Forecast, Month by Month



- ▶ Actual, “no change” and the ten-seed mean over the 79 test months: three lines, one path.
- ▶ Every forecast is essentially last month’s level. The level is persistent; the change is not.
- ▶ The gaps in the table are hundredths of a point, invisible at this scale.
- ▶ **That the three lines are indistinguishable is itself the result.**

Source: Session 2 lab notebook, block C; FRED series DGS10, month-end, downloaded 20 Sep 2026.

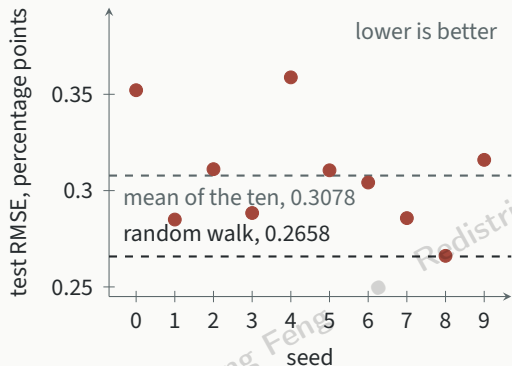
When Did the Network Lose?



Source: Session 2 lab notebook, block C.

- ▶ Each month's squared error minus the random walk's, added up: above zero the model is losing.
- ▶ At the end of the window: AR(1) -0.046 , OLS $+0.858$, MLP $+1.319$.
- ▶ The worst month alone is April 2022, excess 0.281 : 21% of the network's whole loss.
- ▶ **One RMSE over 79 months cannot say when a model lost. This can.**

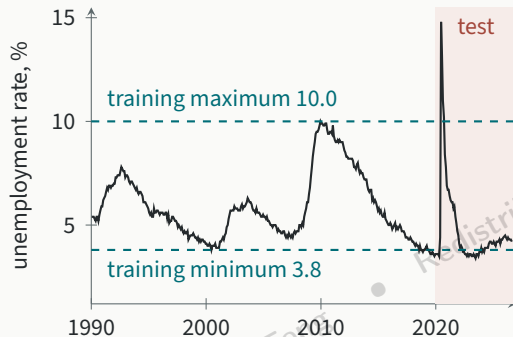
Ten Seeds: a Network Is a Distribution of Answers



- ▶ Only the initial weights differ, but the loss is not convex: the draw selects *which* stationary point you reach. Ten functions, not noise.
- ▶ The seed range is 0.0927; the whole gap from random walk to network is 0.0297, $3.1\times$ smaller.
- ▶ The **ensemble** — average the ten forecasts, then score — gives 0.2956 and still loses.
- ▶ **One seed could have “shown” a win. Report the range and the ensemble.**

Source: Session 2 lab notebook, block C: ten seeds of one width-4 network, 29 parameters, 300 monthly observations.

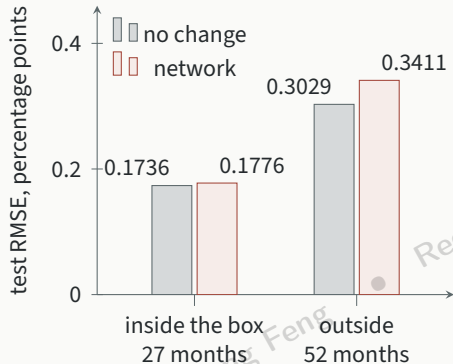
Outside the Training Support



- ▶ Unemployment is outside its training range in 37% of test months: above 10.0 in 2020, below 3.8 later.
- ▶ Likewise the spread (24%), the yield (22%), inflation (18%) and the funds rate (4%).
- ▶ Jointly it is worse: 66% of months are outside on some predictor; there the network extrapolates, as at $w = 30$.
- ▶ **Check the support before believing a flexible model; the next slide splits the errors by it.**

Source: Session 2 lab notebook, block C; FRED series UNRATE, downloaded 20 Sep 2026.

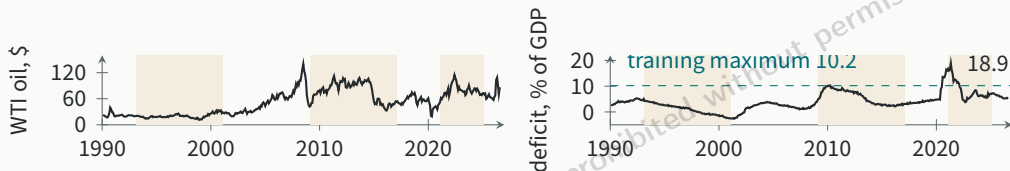
It Loses Where It Extrapolates



- Split the 79 test months by whether every predictor lies inside its 1990–2014 range.
- Inside, the network’s excess over “no change” is $+0.0040$; outside, $+0.0382$, which is $9.5\times$ larger.
- A month inside every marginal range can still be outside the joint one: 66% are, against 37% for unemployment.
- **The network does not lose everywhere. It loses where it has never been.**

Source: Session 2 lab notebook, block C.

More Information, Measured



- ▶ Four added, all dated $\leq t$: **oil** (month-end WTI, a market price, dated t), the federal **deficit** (12-month rolling total, % of GDP, dated $t - 1$), and two party dummies.
- ▶ The rolling sum removes the very large April tax seasonality, at the cost of a year of history plus its lag: 438 rows, 299 training months against the 300 above.
- ▶ `pres_dem` (shaded) switches 6 times in 438 months, twice in the 79 test months; `unified` 13 times, 3 in the test window: one switch per 73 months.
- ▶ **Six changes in thirty-six years is six observations, not 438: a dummy constant across a window is a level shift to memorise, not a predictor.**

Source: Federal Reserve Bank of St. Louis, FRED, downloaded 21 September 2026; party control from the public record.

The Result, with Nine Predictors

Model	RMSE	MAE
random walk	0.2658	0.2097
AR(1)	0.2646	0.2089
OLS, five predictors	0.2842	0.2244
OLS, nine predictors	0.3163	0.2500
MLP, five predictors, ensemble of ten seeds	0.2929	0.2271
MLP, nine predictors, ensemble of ten seeds	0.3021	0.2374

- ▶ Same splits and scoring, four more predictors: OLS got 11% worse, the network 3%.
- ▶ Validation chose width 4 with five predictors (MSE 0.0349) and width 16 with nine (0.0403); no width did better, and the share of test months outside the training box rose from 66% to 72%, the deficit outside in 24% of them and oil in none.
- ▶ **More information widened where the model extrapolates, and cost more than it was worth.**

Source: Federal Reserve Bank of St. Louis, FRED, downloaded 21 September 2026; same target, splits and scoring as above.

What Session 5 Inherits: a Frozen, Dated Baseline

- ▶ Two files leave this deck: the frozen panel, and the four models' test-window forecasts, each row stamped with the date the forecast was made and the date of its target.
- ▶ Session 5 turns central-bank text into a numerical signal. A signal dated $\leq t$ is merged on the forecast date and must improve on *these* numbers.
- ▶ The baseline does not move: the frozen panel, its hashes and the four models' forecasts are untouched, and the nine-predictor run is a second experiment beside it, not a replacement. Whatever text adds is measured against RMSE 0.2658, under the same splits and information rule.
- ▶ The bar is high on purpose: “no change” is what a new data source has to beat.
- ▶ **A new signal is worth what it adds to a fixed, honest baseline, and nothing more.**

Source: Session 2 lab notebook, block C.